IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Received 8 October 2025, accepted 30 October 2025, date of publication 4 November 2025, date of current version 13 November 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3628784

RESEARCH ARTICLE

Deep Learning-Based Multi-Class Detection of LSB Steganography in Digital Images

ALANOUD M. ALMHLBDI, NORAH D. ALTOWAIRQI, AREEJ O. ALSHUTAYRI, AND REHAB K. QAROUT

Department of Computer Science and Artificial Intelligence, College of Computer Science and Engineering, University of Jeddah, Jeddah 21589, Saudi Arabia

Corresponding author: Norah D. Altowairqi (naltowairqi0001.stu@uj.edu.sa)

ABSTRACT Identifying hidden payloads in images has become increasingly critical as steganography continues to challenge traditional security measures. This paper introduces a deep learning framework for both the detection (binary classification) and fine-grained classification (multi-class) of steganographic payloads embedded using Least Significant Bit (LSB) techniques. The proposed system distinguishes between benign images and stego images containing five different payload types: HTML, JavaScript, PowerShell, URLs, and Ethereum-related data. To achieve this, we systematically evaluate various architectures, including a custom Convolutional Neural Network (CNN), hybrid CNN-GRU and CNN-LSTM models, and a Vision Transformer (ViT) at different input resolutions using 5-fold cross-validation. Our experiments reveal a critical finding: image resizing significantly degrades detection performance, as subtle LSB artifacts are often corrupted. While our custom CNN model achieved the highest mean cross-validation accuracy (0.9702), the hybrid CNN-GRU model demonstrated superior generalization on the held-out test set and external dataset, achieving a multi-class accuracy of 0.98 on the test set and 0.97 on the external. This result highlights the advantage of combining the CNN's spatial feature extraction with the GRU's ability to model sequential dependencies for robust payload identification on unseen data.

INDEX TERMS Steganography, steganalysis, least significant bit, convolutional neural network, deep learning, cybersecurity.

I. INTRODUCTION

Steganography, The practice of concealing information within innocuous digital media, has become an important technique for covert communication in the modern cyber era [1]. By embedding secret data (e.g., messages or code) into common file formats like images, steganography enables private data transfer over public networks without drawing suspicion [1]. However, the very success of steganography in ensuring confidentiality has also made it a tool for malicious activities [1]. Criminal organizations are reported to hide illicit instructions and malware payloads inside seemingly normal images, creating an "invisible" channel that evades human detection [2]. Indeed, recent incidents have shown that digital images can carry hidden malware configurations and commands, posing a significant threat to cybersecurity [2]. To combat such abuses, the field of steganalysis has emerged, which aims to detect and decode hidden content in media and thereby prevent attackers from misusing steganographic techniques [1]. In today's security landscape, effective steganalysis of images is recognized as a vital component of defense, given the ubiquity of image-sharing and the ease with which hidden data can proliferate undetected [3].Among various steganographic methods, digital images in the spatial domain (such as raw pixel data) are especially common carriers [2]. A particularly widespread technique is the Least Significant Bit (LSB) substitution, which embeds secret payloads by altering the lowest-order bits of pixel intensity values [4]. LSB steganography is popular because it is straightforward to implement and offers high embedding capacity with minimal perceptible distortion in the carrier image [4]. As a result, it has been widely adopted not only by

The associate editor coordinating the review of this manuscript and approving it for publication was Tao Huang.

© 2025 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

benign applications (watermarking) but also by adversaries seeking to exfiltrate data or deliver malware covertly. In fact, modern malware campaigns frequently leverage simple LSB embedding to hide malicious code or instructions within images, precisely because this method introduces no obvious artifacts and avoids triggering conventional security scans [3]. Studies indicate that LSB-based steganography remains one of the most prevalent strategies in real-world attack campaigns, given its efficiency and stealth [3], [4]. These trends underscore the urgency for advanced detection techniques targeting LSB payloads in images as a means of strengthening cybersecurity defenses. Over the years, various image steganalysis methods have emerged to uncover hidden data. Early approaches primarily relied on hand-crafted statistical features and traditional machine learning (ML), using classifiers such as SVMs or ensembles to distinguish between clean and steganographic images [1]. However, these feature-based techniques struggled to generalize across diverse images due to their inability to capture subtle embedding artifacts [1]. Recently, researchers have shifted toward deep learning (DL), particularly convolutional neural networks (CNNs), which automatically learn rich feature representations highlighting subtle payload-induced perturbations. CNN-based steganalyzers have demonstrated superior detection performance, significantly improving accuracy even at low embedding rates and becoming the cornerstone of modern steganalysis [1], [5].Despite these advances, several gaps remain. Most steganalysis methods focus on binary classification (identifying presence or absence of hidden data) without recognizing the specific steganographic technique or payload [2]. Additionally, robustness and generalization remain significant concerns, as deep learning models frequently exhibit performance degradation under common image transformations or unseen steganographic schemes [6]. Finally, deep learning-based steganalysis models often lack interpretability, functioning as closed box with little transparency regarding decision-making processes. This opacity can limit trust and adoption in sensitive cybersecurity applications, highlighting crucial challenges that need addressing to enhance steganalysis efficacy [7]. In response, this study introduces a deep neural network framework to detect and classify LSB-based hidden payloads using advanced deep learning techniques. Our approach moves beyond binary detection, classifying payload types and aiming for robust generalization across diverse images and conditions. Furthermore, we initiate steps toward interpretability, addressing critical gaps identified in existing literature. To provide a comprehensive solution, this research tackles the problem from two distinct but related perspectives. First, we frame it as a binary classification task to determine the mere presence of a hidden payload (a stego vs. clean image), which is crucial for rapid threat detection. Second, we address it as a more challenging multi-class classification task to identify the specific type of embedded payload. This dual approach not only detects a threat but also provides valuable forensic intelligence about its nature. Our research provides significant contributions to steganalysis by presenting a deep learning-based system capable of precise detection and classification of LSB payloads. We systematically evaluate and compare advanced architectures, including custom CNN, hybrid CNN-RNN models (CNN-LSTM, CNN-GRU), and Vision Transformer (ViT). Our solution enhances steganalysis accuracy by distinguishing clean images from those containing payloads such as HTML, JavaScript, PowerShell, URLs, and Ethereum data. Notably, the hybrid CNN-GRU model achieved perfect accuracy and F1-score, demonstrating the advantage of combining CNN spatial extraction with RNN sequential analysis. Additionally, the paper emphasizes preserving image fidelity during preprocessing and developing lightweight models for practical cybersecurity applications, laying foundations for robust security solutions. The remainder of this paper is structured as follows: Section II discusses related work and relevant methodologies in steganalysis. Section III describes our proposed methodologies, detailing model architectures and training procedures. Section IV presents experimental results, highlighting model performance and comparative analyses. Finally, Section V concludes with insights, limitations, and potential directions for future research.

II. RELATED WORK

Deep learning has rapidly become the cornerstone of modern steganalysis, achieving superior detection accuracy compared to earlier hand-crafted feature methods [8]. Convolutional neural networks (CNNs) in particular enable automatic extraction of subtle steganographic artifacts. A variety of CNN architectures have been proposed to detect hidden data in images, often by incorporating domain-specific preprocessing to amplify embedding noise. For example, Ntivuguruzwa et al. designed a spatial domain CNN with a high-pass filter bank (Spatial Rich Model filters) and depthwise separable convolutions to enhance hidden signal-to-noise ratio [1]. Their model improved detection accuracy by up to 5–10% (and as high as 17% at low embed rates) over prior CNNs on content-adaptive algorithms like WOW and S-UNIWARD [1]. Fu et al. similarly introduced a lightweight CNN (''CCNet'') using channel-attention mechanisms and convolutional pooling to better emphasize informative residual features, achieving state-of-the-art accuracy on spatial LSB payloads [8].

Beyond standard CNN classifiers, other deep learning strategies have been explored. Some works employ autoencoder and anomaly detection principles, for instance, Ayaluri et al. developed a noise-aware deep autoencoder that removes natural image noise and focuses on non-Gaussian embedding artifacts [9]. By denoising inputs at multiple stages, their model improved detection efficiency while reducing computational overhead [9]. Generative models have also been leveraged: one study designed an adversarially trained steganalyzer (RS-GAN) that reconstructs a suspected image through a GAN learned on cover data, then flags

anomalies by comparing reconstruction errors [10]. This method proved robust against adversarial steganography attempts, e.g. It maintained 72% accuracy on adversarial examples that crippled a conventional CNN to 47% [10]. Another emerging direction is reducing the dependence on large labeled datasets. Li et al. combined active learning with deep reinforcement learning to intelligently select the most informative images to label [2]. Using this strategy alongside differential evolution hyper-parameter tuning, they achieved over 93% F-score on BOSSBase with only a fraction of the training data typically required [2]. This addresses a common limitation in deep steganalysis, the need for extensive cover/stego training corpora.

While deep models dramatically improve detection performance, several limitations persist in prior work. Most deep steganalyzers are formulated as binary classifiers (cover vs. stego) and do not identify the steganographic payload type or algorithm used, an aspect largely overlooked in the literature. They also tend to target one embedding domain at a time and may not generalize across steganography schemes. For instance, a network trained on spatial-LSB steganography often struggles to detect frequency domain or other scheme variants. Robustness is another concern: many models are brittle under even mild image distortions. Alrusaini et al. recently showed that common CNN-based detectors (Xu-Net, Yedroudj-Net, etc.) suffer significant accuracy drops under resizing, noise, or recompression, underscoring the need for more resilient architectures [11]. The interpretability of deep steganalysis is yet another open challenge, most networks operate as closed box, offering little insight into why an image is flagged. Initial attempts at explainable steganalysis (e.g. the STEG-XAI framework) have demonstrated that integrating explainable AI techniques can highlight which regions or features led to a detection decision [7], but such approaches are still in their infancy.

Beyond core vision tasks, ViTs have been explored in application domains such as finance, where time-series are converted to image-like representations and fed to ViT/DeiT/Swin models. For example, the results by Gezici and Sefer [12] and Tuncer et al. [13] support the use of ViTs when high-frequency details and long-range dependencies matter, and they motivate our ViT comparisons and robustness checks. In summary, recent work showcases an array of deep learning techniques from novel CNN variants and attention-based models to autoencoders, GANs, and reinforcement learning, that markedly advance image steganalysis. These studies primarily focus on improving binary detection accuracy under ideal conditions. The present work distinguishes itself by addressing some gaps noted above: we concentrate on classifying steganographic payloads (not just detecting their presence), with an emphasis on LSB-based methods. By doing so, we extend deep steganalysis beyond binary classification, aiming to determine which type of payload or embedding scheme is used. We also prioritize robustness and generalization in our design. This approach pushes the field toward more forensic insight, not only catching that an image contains hidden data, but also shedding light on the nature of the payload, thereby filling an important void in the existing research. Each of these contributions builds on the limitations of prior work to advance the capabilities of steganographic payload detection and classification.

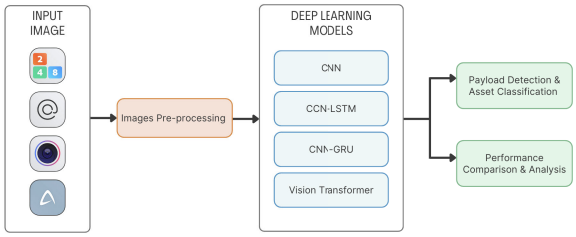

FIGURE 1. Main workflow.

III. METHODOLOGY

Our proposed methodology implements a DL framework for robust steganography analysis, focusing on both the detection of hidden payloads and the classification of the malicious asset type. The core of our approach is a comparative evaluation of distinct neural network architectures to identify the most effective model for this nuanced task. Figure 1 presents an overview of the system architecture, showcasing the parallel evaluation pipeline. The first component of our approach uses Convolutional Neural Networks (CNNs). We train a custom CNN architecture from scratch to learn discriminative spatial features directly from the pixel domain, capturing subtle statistical patterns and noise introduced by steganographic embedding. This CNN acts as a feature extractor and classifies whether an image contains hidden data [14], [15]. We also evaluate hybrid CNN–RNN models (CNN–LSTM, CNN–GRU). The CNN backbone extracts a feature map that we reshape into a spatial sequence ($H \times W$, C) in raster order; the recurrent layers then model long-range spatial correlations to capture subtle artifacts introduced by LSB modifications [1]. In addition, we investigate the Vision Transformer (ViT) architecture, which differs from CNNs by dividing the image into fixed-size patches and applying a transformer encoder [16]. This self-attention mechanism enables the model to learn global relationships across the entire image, making it well-suited for detecting faint statistical deviations characteristic of advanced steganography [17]. We compare the performance of the CNN, CNN-LSTM, CNN-GRU, and ViT models to assess their effectiveness in detecting and classifying malicious payloads, using a carefully prepared dataset of clean and stego images. Hyperparameters for each model were tuned to optimize performance, and the results are presented in the following sections.

Our main work starts with raw images passed to preprocessing before being fed to our models, which will be trained/fine-tuned in order to achieve the best possible results.

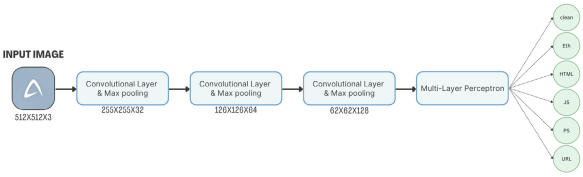

FIGURE 2. Our custom CNN architecture.

A. CLASSIFICATION MODELS

The classification models are designed to categorize malicious asset types into six groups: "clean," "JavaScript code," "Obfuscated JavaScript in HTML," "PowerShell scripts," "Ethereum Addresses," and "URL and IP Addresses." To identify the most effective classification model, four different models were employed: Custom CNN, CNN-LSTM, CNN-GRU, and pre-trained ViT. Figure 2 illustrates the classification process used in this study. The methodology is consistent across all four models, with the only variation being the model itself. The process involves reading and processing the data, training the chosen model, and ultimately evaluating its performance.

1) CUSTOM CNN

Our custom Convolutional Neural Network (CNN) is built from several key layers that process images in stages. The main components include:

- **Convolutional Layer:** The core of the CNN, responsible for identifying local features like edges, corners, and textures by applying a filter (e.g., 3×3 grid) over the input image. This creates a feature map that highlights specific image features. The convolution operation is defined as:

$$(f * I)(x, y) = \sum_{c} \sum_{i} \sum_{j} f_c(i, j) \cdot I_c(x + i, y + j) + b \tag{1}$$

  where I is the input image patch, f is the filter, b is the bias, and c represents the channel. Our CNN has three convolutional layers with 32, 64, and 128 filters to progressively capture more complex image features.
- **Pooling Layer:** Max pooling decreases the spatial size of feature maps, simplifying the representation and providing robustness against minor variations in the input. The max pooling operation is:

$$P(R_k) = \max_{(i,j) \in R_k} A_{ij} \tag{2}$$

  where A is the input feature map, and R_k is the k-th pooling region.
- **Fully Connected (Dense) Layer:** Following feature extraction, the resulting feature maps are converted into a single vector through flattening, which is then processed by one or more dense layers to identify global patterns. The output of a neuron is calculated as:

$$y = \phi(W \cdot x + b) \tag{3}$$

  where x denotes the input vector, W the weight matrix, b the bias vector, and ϕ the activation function.
- **Activation Functions:** Activation functions introduce non-linearity into neural networks, enabling the model to learn intricate patterns and data relationships.
  1) **ReLU (Rectified Linear Unit):** Used to add non-linearity and reduce the impact of vanishing gradients.

$$f(x) = \max(0, x) \tag{4}$$

  2) **SoftMax:** Used in the final layer to convert raw output scores into class probabilities.

$$\text{Softmax}(x_i) = \frac{e^{x_i}}{\sum_{j=1}^{c} e^{x_j}} \tag{5}$$

  where c is the total number of classes.
  Our CNN architecture begins with the input image, which is processed through multiple convolutional layers. The extracted features are then passed to a classification head (MLP), which performs the final image classification.

2) CNN-LSTM MODEL

This model merges the spatial feature extraction ability of a CNN with the sequence-processing strength of a LSTM network. The CNN-generated feature maps are passed into the LSTM layer, which interprets the spatial features as a sequence. This allows the model to effectively capture long-range relationships among features throughout the image.

At the heart of the LSTM is the cell state (C_t), which is controlled by three gates: the forget gate (f_t), input gate (i_t), and output gate (o_t). These gates regulate information flow, enabling the network to retain or discard information across long sequences.

The core LSTM operations are defined as:

- **Forget Gate:**

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \tag{6}$$

- **Input Gate:**

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \tag{7}$$

- **Output Gate:**

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \tag{8}$$

- **Cell State Update:**

$$C_t = f_t \odot C_{t-1} + i_t \odot \tanh(W_c \cdot [h_{t-1}, x_t] + b_c) \tag{9}$$

- **Hidden State:**

$$h_t = o_t \odot \tanh(C_t) \tag{10}$$

Our CNN-LSTM model uses two CNN layers with 32 and 64 filters, followed by a bidirectional LSTM layer with 64 units to process the extracted features from both forward and backward directions.

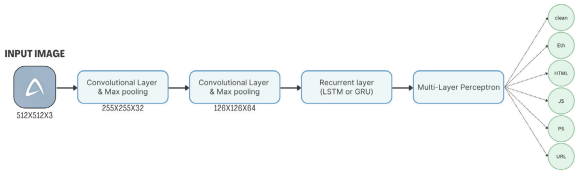

FIGURE 3. Our hybrid CNN-RNN.

3) CNN-GRU MODEL

Like the CNN-LSTM, the CNN-GRU is a hybrid model that incorporates a Gated Recurrent Unit (GRU) layer to process the features extracted by the CNN. The GRU simplifies the LSTM by combining the forget and input gates into a single *update gate* and merging the cell state with the hidden state. This design makes the GRU more computationally efficient while maintaining similar performance across many tasks.

The GRU uses two gates: the reset gate(r_t) and the update gate (z_t).

- **Reset Gate:**

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t] + b_r) \tag{11}$$

- **Update Gate:**

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t] + b_z) \tag{12}$$

- **Candidate Hidden State:**

$$\tilde{h}_t = \tanh(W_h \cdot [r_t \odot h_{t-1}, x_t] + b_h) \tag{13}$$

- **Hidden State:**

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \tag{14}$$

Our CNN-GRU model is structured identically to the CNN-LSTM, with two CNN layers (32 and 64 filters) followed by a bidirectional GRU layer with 64 units as presented in Figure 3.

Our hybrid model starts by processing the input image through two convolutional layers to extract spatial features. These features are then fed into a recurrent layer, either LSTM or GRU, to capture sequential dependencies. Finally, the output is passed to a classification head (MLP) that performs the image classification.

4) VISION TRANSFORMER (ViT)

The Vision Transformer (ViT) employs the Transformer architecture, traditionally used in NLP, for image classification tasks. Unlike CNNs, ViT represents images as sequences of smaller image patches [16]. Key steps include:

- **Image Patching and Embedding:** The image is divided into non-overlapping patches, which are then flattened and projected into an embedding space:

$$z_0 = [x_{\text{class}}; E_{p_1}; E_{p_2}; \ldots; E_{p_N}] + E_{\text{pos}} \tag{15}$$

where p_i are the flattened patches, E is a linear projection (embedding matrix), and a learnable [class] token is prepended.

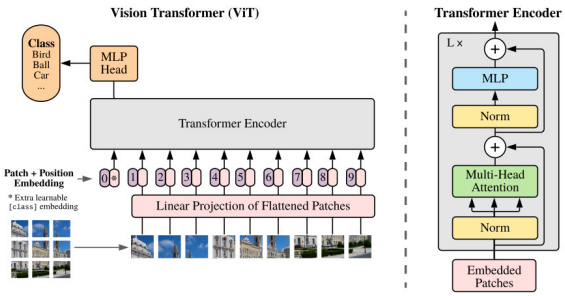

FIGURE 4. ViT architecture from the original paper [16].

- **Positional Encoding:** Since ViT lacks inherent knowledge of patch locations, positional embeddings E_{pos} are added to retain spatial information.
- **Transformer Encoder:** These embedding sequences are processed through multiple Transformer blocks, where Multi-Head Self-Attention captures global relationships among image patches. The attention mechanism is:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) \tag{16}$$

where Q (Query), K (Key), and V (Value) are matrices derived from the input embeddings.
- **Classification:** The final output state corresponding to the [class] token is passed to an MLP for classification.

We employ a pre-trained `ViT-base- patch16-224-in21k` model, adapting it by replacing the final classification layer to match the number of classes in our dataset.

The ViT model begins by dividing the input image into a series of equally sized patches. Each patch undergoes linear embedding and is combined with positional encodings before entering the transformer encoder. Through self-attention mechanisms, the encoder models global relationships across all patches. The resulting representation is then passed to a classification head (MLP), which performs the final image classification as shown in Figure 4.

B. STEGANOGRAPHY DETECTION

Using the acquired dataset, we conducted an evaluation of the models on a binary classification task aimed at steganography detection, distinguishing between clean and non-clean instances. To facilitate this, the original multi-class classification labels and predictions were systematically converted into binary form according to the following rule:

$$y_{\text{detection}} = \begin{cases} \text{clean}, & \text{if } y_{\text{classification}} = \text{clean} \\ \text{not clean}, & \text{otherwise} \end{cases} \tag{17}$$

The model's performance was evaluated using the same metrics used earlier with the multi-class classification task, providing consistency in comparative analysis.

C. EVALUATION METRICS

To evaluate classification model performance, we use four standard metrics: precision, recall, F1-score, and accuracy, as well as a confusion matrix.

1) CONFUSION MATRIX

A confusion matrix provides detailed insights into the performance of classification by showing how often each class is correctly predicted and how frequently it is misclassified as another. It allows us to identify which classes are being confused with others.

It shows four main values:

- **TP (True Positive)**: Correct prediction of the positive class.
- **TN (True Negative)**: Correct prediction of the negative class.
- **FP (False Positive)**: Incorrectly predicted as positive.
- **FN (False Negative)**: Incorrectly predicted as negative.

2) PRECISION

Precision measures the reliability of the model's positive predictions.

$$\text{Precision} = \frac{TP}{TP + FP} \tag{18}$$

A high precision means the model rarely predicts a positive case incorrectly, indicating a low false-positive rate.

3) RECALL

Recall, also known as sensitivity, evaluates the model's capability to detect all actual positive instances.

$$\text{Recall} = \frac{TP}{TP + FN} \tag{19}$$

A high recall suggests the model misses few true positives.

4) F1-SCORE

The F1-score, calculated as the harmonic mean of precision and recall, provides a balanced measure of the model's effectiveness in predicting positive cases.

$$\text{F1-score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \tag{20}$$

A high F1-score indicates that the model is both accurate in its positive predictions and effective at finding all true positives, minimizing both types of errors.

5) ACCURACY

Accuracy reflects the overall proportion of correct predictions across all classes.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{21}$$

Although accuracy provides useful information, it can be deceptive when class distributions are imbalanced. Thus, precision, recall, and F1-score are additionally reported to deliver a more thorough assessment of model performance.

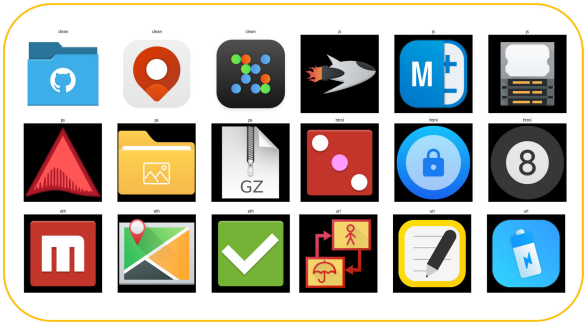

FIGURE 5. Random samples from the dataset [14].

IV. RESULTS

This section details the experimental setup, introduces the dataset, and presents the performance evaluation of the proposed DL framework for steganography detection and classification.

A. DATASET

In our experiments, we used the "Stego-Images-Dataset," which was created by Cassavia et al. [14], consisting of 32,000 high-resolution icon images (512 × 512 pixels) that simulate real-world scenarios where attackers embed various malicious payloads using steganography. These icons represent common digital assets found across platforms including mobile devices, tablets, set-top boxes, and desktops, and are typically used to display applications or file types. The dataset includes stego images containing realistic malicious payloads, reflecting threats seen in Advanced Persistent Threats (APTs) and crimeware toolkits, making it suitable for evaluating steganographic detection and classification models.

The stego images' labels contain the following classes:

- **Clean:**Benign icon images with no embedded data, serving as control samples to assess model specificity and false-positive rates.
- **JavaScript code (JS):**Cross-platform attack scripts that fetch additional payloads, de-obfuscate malicious files, or enable fileless execution.
- **Obfuscated JavaScript in HTML (HTML):**Scripts concealed within text or comments using various encoding techniques, complicating web-based detection and analysis.
- **PowerShell scripts (PS):**Malicious PowerShell hidden via steganography (e.g., Invoke-PSImage), employed in campaigns against the Pyeongchang Olympics, Greystars, and Bandook.
- **Ethereum addresses (Eth):**Fixed-length wallet hashes embedded to direct ransomware payments or crypto-jacking proceeds, enabling covert C&C updates.
- **URLs and IP addresses (URL):**Communication end-points for data exfiltration or payload retrieval, now hidden in images to evade network and binary analysis.

Figure 5 presents some random samples from the dataset used.

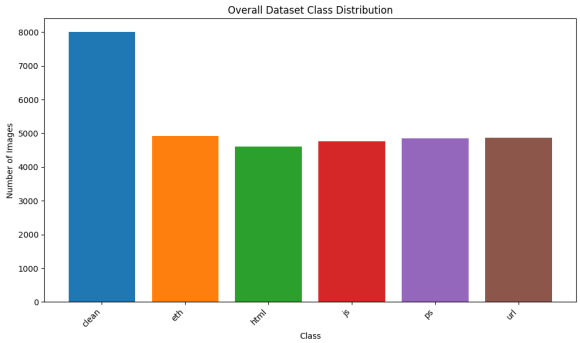

FIGURE 6. Class distribution for the dataset.

FIGURE 7. Class distribution for the Train, Validation, and Test sets.

The dataset comprises 8,000 equally sized images drawn from GPL-3-licensed open repositories. To simulate realistic attack scenarios, each clean image was randomly paired with three payloads selected with equal probability from five categories (JavaScript code, obfuscated JavaScript in HTML, PowerShell scripts, Ethereum addresses, and URLs/IPs), and embedded using the least significant bit (LSB) technique. LSB steganography allows altering the least significant bit of the color space of each pixel to cloak arbitrary information. A variant of the LSB technique, exploiting only two color channels, has become one of the most popular approaches observed in real attacks to conceal malicious payloads in images. This method is commonly observed in real-world steganographic steganographic threats [18].This procedure produced 32,000 stego samples, with the following labels (see Figure 6):

1) **Clean:** 8000 samples.
2) **JS:** 4765 samples.
3) **HTML:** 4613 samples.
4) **PS:** 4845 samples.
5) **Eth:** 4913 samples.
6) **URL:** 4864 samples.

The images are divided into training, validation, and test sets (see Figure 7), with the following ratio respectively (80%, 10%, 10%).The test set contains only images unseen during training and includes all five payload types. By design, clean images constitute the minority class in the detection task, allowing us to evaluate detection under class imbalance and to perform fine-grained classification of malicious payloads.

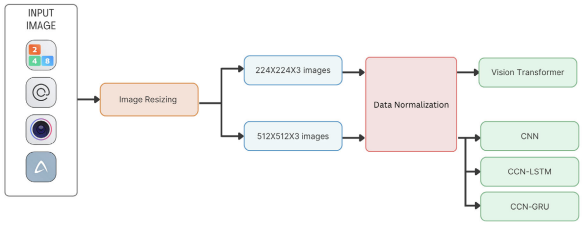

FIGURE 8. Preprocessing workflow.

TABLE 1. Mean 5-Fold cross-validation accuracy.

Model	Image Size	Mean Accuracy
CNN	512×512	0.9702
CNN	224×224	0.6562
CNN-GRU	512×512	0.9526
CNN-GRU	224×224	0.4348
CNN-LSTM	512×512	0.8145
CNN-LSTM	224×224	0.4630

B. DATASET PREPROCESSING

Before being fed into the deep learning models, the images underwent a preprocessing pipeline as shown in Figure 8. A key variable in our experimentation was image resolution. To investigate its impact on performance, two sets of inputs were prepared:

1) **Full-Resolution Images** (512 × 512)**:** Images were kept at their original 512 × 512 pixel resolution for the CNN, CNN-GRU, and CNN-LSTM models.
2) **Downsampled Images** (224 × 224)**:**Images were resized to 224 × 224 pixels. This was a requirement for the pre-trained ViT model and was also used to train the other models for a comparative analysis of the effect of resizing.

Following resizing, all pixel values were normalized to a range of [0,1] by dividing by 255. This ensures numerical stability and consistent feature scaling during training.

C. EXPERIMENTAL SETUP AND TRAINING

All models were trained using a 5-fold cross-validation strategy to ensure robust and generalizable performance evaluation. Due to its high computational cost, 5-fold CV could not be applied to the ViT model. For each fold, models were trained for 10 epochs using a batch size of 128. We employed the Adam optimizer with a learning rate of 1e-3 and a ReduceLROnPlateau callback, which decreases the learning rate by a factor of 0.2 if the validation loss does not improve for 3 consecutive epochs, down to a minimum of 1e-6.

D. CROSS-VALIDATION AND TEST SET RESULTS

The 5-fold cross-validation results, shown in Table 1, clearly indicate that models trained on full-resolution 512 × 512 images consistently outperformed their counterparts trained on downsampled 224×224 images. The Custom CNN (512 × 512) achieved the highest mean accuracy, establishing it as our best-performing model.

Because five-fold cross-validation was computationally prohibitive for the ViT model, we report its results

TABLE 2. Classification results (multi-class classification).

Model	Accuracy	Precision	Recall	F1-score
CNN 512	0.97	0.97	0.97	0.97
CNN 224	0.67	0.77	0.64	0.59
CNN-GRU 512	0.98	0.98	0.98	0.98
CNN-GRU 224	0.56	0.45	0.51	0.45
CNN-LSTM 512	0.93	0.93	0.93	0.93
CNN-LSTM 224	0.40	0.42	0.34	0.29
ViT 224	0.97	0.97	0.97	0.97

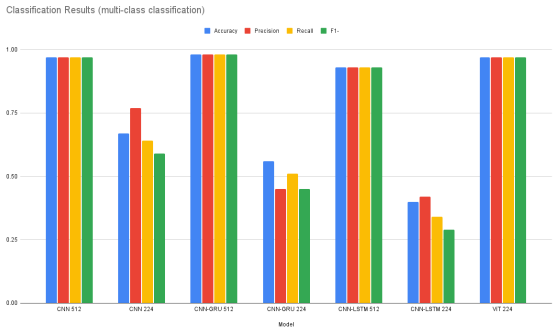

FIGURE 9. Classification Results (multi-class classification).

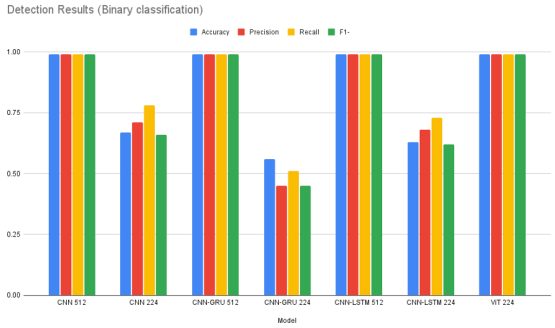

FIGURE 10. Detection Results (Binary classification).

TABLE 3. Detection results (binary classification).

Model	Accuracy	Precision	Recall	F1-score
CNN 512	0.99	0.99	0.99	0.99
CNN 224	0.67	0.71	0.78	0.66
CNN-GRU 512	0.99	0.99	0.99	0.99
CNN-GRU 224	0.56	0.45	0.51	0.45
CNN-LSTM 512	0.99	0.99	0.99	0.99
CNN-LSTM 224	0.63	0.68	0.73	0.62
ViT 224	0.99	0.99	0.99	0.99

alongside the other models in Tables 2 and 3. Although the CNN performed best under cross-validation, the CNN-GRU (512×512) achieved superior performance on the final held-out test set for the primary multi-class classification task. We therefore consider the CNN-GRU our best-performing model. For completeness, we present detailed test results for both the CNN-GRU and CNN trained on 512×512 images.

1) DETAILED TEST RESULTS FOR THE BEST MODEL: **CNN-GRU**(512 × 512)

See Tables 3, 4, and 5. Figures 11 and 12.

TABLE 4. Multi-class Classification Report for CNN-GRU 512 × 512.

Class	Precision	Recall	F1-score
Benign	0.98	1.00	0.99
eth	1.00	0.99	0.99
html	0.98	0.94	0.96
js	0.94	0.99	0.96
ps	1.00	0.98	0.99
url	1.00	0.99	0.99

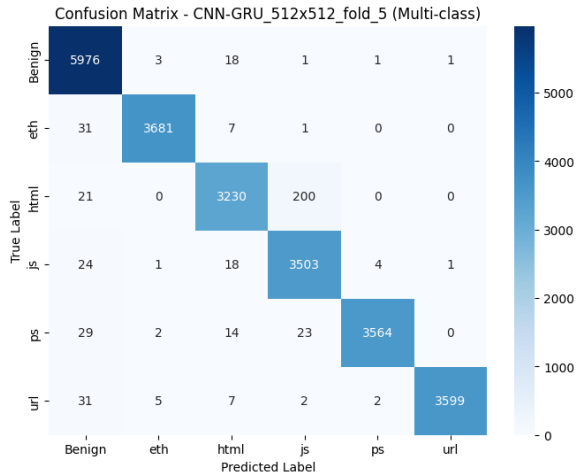

FIGURE 11. Confusion Matrix (Multi-class) for CNN-GRU 512 × 512.

TABLE 5. Binary Detection Report (Benign vs. Not Benign) for CNN-GRU 512 × 512.

Class	Precision	Recall	F1-score
Benign	0.98	1.00	0.99
Not Benign	1.00	0.99	1.00

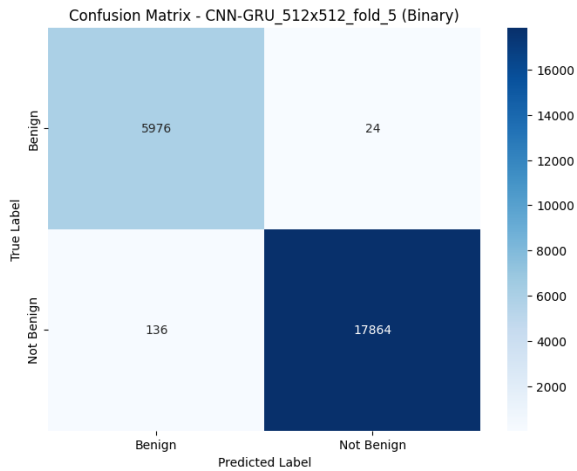

FIGURE 12. Confusion Matrix (Binary) for CNN-GRU 512 × 512.

2) COMPARATIVE TEST RESULTS: **CNN**(512 × 512)

The experimental results clearly identify the CNN-GRU model as the most effective architecture on the test set, demonstrating a superior capacity for both detecting and classifying steganographic payloads. The custom CNN also delivered outstanding results, underscoring the strength of spatial feature extraction in identifying the subtle artifacts introduced by steganography. Conversely, the Vision

TABLE 6. Multi-class Classification Report for CNN 512 × 512.

Class	Precision	Recall	F1-score
Benign	1.00	1.00	1.00
eth	1.00	1.00	1.00
html	0.93	0.90	0.92
js	0.89	0.91	0.90
ps	0.97	0.99	0.98
url	1.00	1.00	1.00

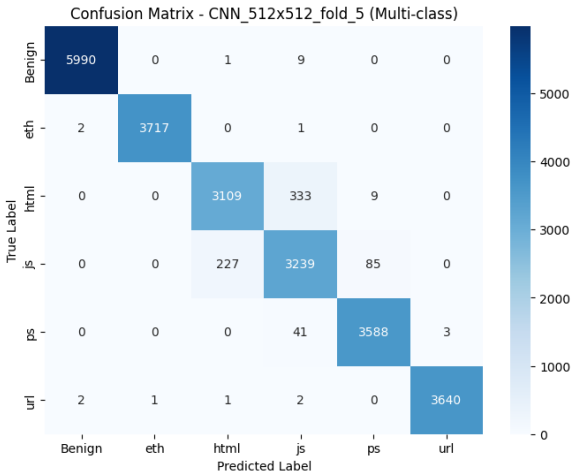

FIGURE 13. Confusion Matrix (Multi-class) for CNN 512 × 512.

TABLE 7. Binary Detection Report (Benign vs. Not Benign) for CNN 512 × 512.

Class	Precision	Recall	F1-score
Benign	0.99	0.99	0.99
Not Benign	0.99	0.99	0.99

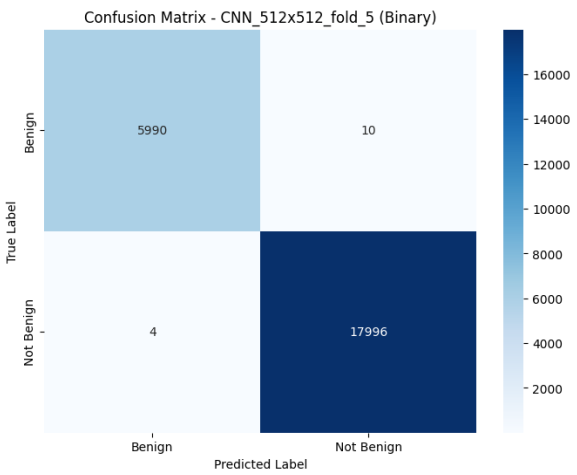

FIGURE 14. Confusion Matrix (Binary) for CNN 512 × 512.

Transformer (ViT), while still performing strongly, showed slightly reduced performance due to the necessary downsampling of images from their original 512×512 resolution to 224×224 to fit the pre-trained model's requirements. This resizing likely caused a loss of the very high-frequency, subtle details that are critical for steganalysis, a challenge the convolutional models avoided by processing the full-resolution images. This highlights the importance of preserving image fidelity for this specific task.

FIGURE 15. Samples of the external dataset.

TABLE 8. CNN 512 × 512 Results (External Dataset).

Class	Precision	Recall	F1-score
Benign	0.75	0.04	0.07
Not Benign	0.48	0.99	0.65

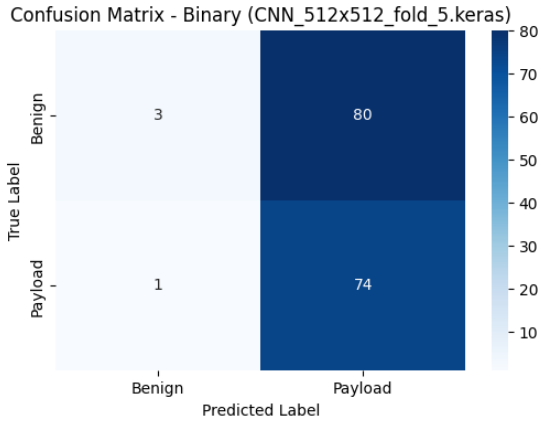

FIGURE 16. Confusion Matrix (Binary) for CNN 512 × 512 (External Dataset).

E. GENERALIZATION ON AN EXTERNAL DATASET

To assess the real-world applicability and generalization capability of our top models, we conducted an experiment on an external dataset. We sourced clean, noise-free 512 × 512 images from a subset of the "TAMPERE17" database [19], and used an LLM-generated script to embed LSB payloads. The results starkly highlight the superior generalization of the CNN-GRU model.

1) CNN 512 × 512 RESULTS (EXTERNAL DATASET)

See Tables 8 and 9. Figure16 and 17.

2) CNN-GRU 512 × 512 RESULTS (EXTERNAL DATASET)

The stark contrast in performance on this external dataset is highly informative. While both models performed well on the original test set, the CNN's failure to generalize suggests it may have overfitted to subtle, dataset-specific artifacts present in the original "Stego-Images-Dataset." In contrast, the CNN-GRU's near-perfect performance indicates that it learned the more fundamental and universal patterns of LSB embedding, allowing it to successfully identify steganography in images from an entirely different source and embedding process. This result provides compelling evidence for the CNN-GRU's superior robustness and real-world applicability.

F. DISCUSSION RESULTS

Our experimental results offer several key insights into deep learning-based steganalysis, demonstrating a significant advancement over existing approaches. The most significant

finding is the critical impact of image resolution on model performance. Across all architectures, models trained on full-resolution 512 × 512 images drastically outperformed those trained on downsampled images. This is clearly illustrated by the performance of the Vision Transformer (ViT) model. Despite being a state-of-the-art architecture, its performance, while strong, lagged behind our convolutional models because its input requirements necessitated downsampling images to 224 × 224. This resizing process inevitably corrupts or destroys the fragile, high-frequency patterns of LSB steganography. As noted by Ghadi et al. [20], such transformations pose a fundamental challenge to data hiding. Our results empirically confirm this, underscoring a crucial guideline for future research: preserving original image fidelity is paramount for effective LSB steganalysis. This focus on high-fidelity feature extraction helps explain our models' superior performance compared to previous benchmarks. For instance, Cassavia et al. [14] achieved a commendable detection accuracy of 0.992 and a classification accuracy of 0.820. In contrast, our proposed CNN-GRU, custom CNN, and CNN-LSTM models all achieved near-perfect scores of 0.99 across all detection metrics (accuracy, precision, recall, and F-Measure). In the more challenging multi-class classification task, our CNN-GRU achieved a top-performing accuracy and F1-score of 0.98. Our other convolutional models also demonstrated strong results, with metrics ranging from 0.95 to 0.98, a substantial improvement over the 0.829 F-Measure reported in the prior work. This enhanced capability stems from the proficiency of convolutional architectures in spatial feature extraction, which is critical for identifying the subtle alterations introduced by steganography, a principle supported by existing literature [21], [22]. An equally important finding is the divergence between cross-validation and test set performance, which highlights the superior generalization of the hybrid CNN-GRU model. While a simpler CNN architecture achieved high accuracy across validation folds, the CNN-GRU generalized more effectively to the unseen test set. This suggests that while the CNN was adept at learning patterns within the training data, the GRU layer provided an additional layer of robustness. By treating the CNN's spatial features as a sequence, the GRU models contextual relationships across the image, allowing it to better identify payload patterns in novel data. The experiment on the externally generated "TAMPERE17" dataset provides the most compelling evidence for the CNN-GRU's robustness. While the standard CNN model's performance collapsed completely on this new data (0.49 accuracy), the CNN-GRU maintained a remarkable accuracy of 0.97. This stark divergence proves that the CNN-GRU did not merely memorize characteristics of the training data but learned the fundamental, generalizable patterns of LSB embedding. This robustness is critical for any real-world security application, where a model will inevitably encounter images from diverse and unforeseen sources. Given its performance, our proposed CNN-GRU model offers a high-performance solution for operational cybersecurity systems. While it carries a slightly higher computational cost, its superior generalization makes it a more reliable choice for deployment. Such a model could be integrated into:

- Network Intrusion Detection Systems (NIDS): To scan image files in real-time as they traverse a network.
- Digital Forensics Toolkits: As a post-mortem analysis tool to scan seized devices for hidden data.
- Endpoint Security Solutions: To analyze images on a user's device, preventing the execution of hidden malware.

The model's ability to perform both binary detection and multi-class classification provides tiered value; rapid detection can trigger an immediate alert, while subsequent classification can inform analysts about the threat's nature (e.g., a PowerShell script versus a URL), enabling a more targeted response. However, like many in the field, we acknowledge the need for future work. As discussed by Cassavia et al. [14] and others [15], [23], dataset expansion to include a broader range of steganographic algorithms and image types is crucial for enhancing generalizability. Furthermore, exploring the interpretability of these models could provide deeper insights into their detection mechanisms, leading to more robust and explainable solutions.

TABLE 9. CNN-GRU 512 × 512 Results (External Dataset).

Class	Precision	Recall	F1-score
Benign	1.00	0.94	0.97
Not Benign	0.94	1.00	0.97

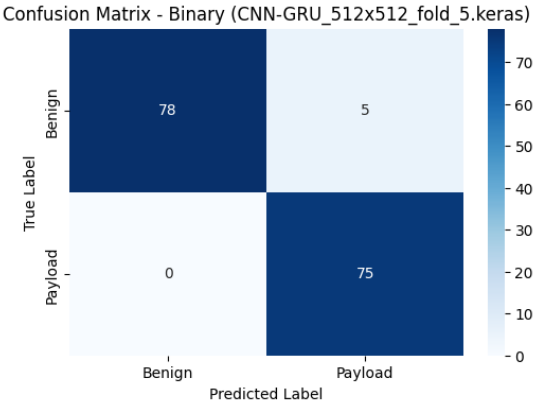

FIGURE 17. Confusion Matrix (Binary) for CNN-GRU 512 × 512 (External Dataset).

V. CONCLUSION

This paper introduces a deep learning framework for the detection and multi-class classification of LSB steganographic payloads. Our key findings demonstrate that preserving original image resolution is critical for effective steganalysis, as downsampling significantly degrades model performance. While a standard CNN performed best in cross-validation, the hybrid CNN-GRU model proved to be definitively superior, showing better generalization on the held-out test set and, most critically, on an entirely new,

externally generated dataset where the CNN model failed. This highlights the CNN-GRU's ability to learn robust, fundamental features of LSB steganography rather than overfitting to a specific dataset. These results establish the CNN-GRU architecture as a powerful and reliable choice for deployment in real-world cybersecurity systems, offering a robust defense against the covert threat of steganography in digital images. Future work should focus on testing this architecture against a wider array of steganographic algorithms and further enhancing its efficiency for real-time applications.

REFERENCES

[1] J. D. L. C. Ntivuguruzwa and T. Ahmad, "A convolutional neural network to detect possible hidden data in spatial domain images," *Cybersecurity*, vol. 6, no. 1, Sep. 2023.

[2] L. Bohang, N. Li, J. Yang, O. Alfarraj, F. Albelhai, A. Tolba, Z. A. Shaikh, R. Alizadehsani, P. Pławiak, and P. L. Yee, "Image steganalysis using active learning and hyperparameter optimization," *Sci. Rep.*, vol. 15, no. 1, Mar. 2025.

[3] A. Liguori, M. Zuppelli, D. Gallo, M. Guarascio, and L. Caviglione, "A deep learning-based approach for stegomalware sanitization in digital images," *J. Intell. Inf. Syst.*, vol. 2025, pp. 1–24, Apr. 2025.

[4] J. D. Miranda and D. J. Parada, "LSB steganography detection in monochromatic still images using artificial neural networks," *Multimedia Tools Appl.*, vol. 81, no. 1, pp. 785–805, Jan. 2022.

[5] M. Y. A. Mostafa, E. A. Al-Qhtani, A. H. Samak, A. A. Ibrahim, M. Elloumi, and A. H. Ahmed, "Deep learning-based steganalysis for detection and classification of possible hidden content in images," *Fusion, Pract. Appl.*, vol. 17, no. 2, pp. 377–393, 2024.

[6] N. J. D. L. Croix, T. Ahmad, and F. Han, "Comprehensive survey on image steganalysis using deep learning," *Array*, vol. 22, Jul. 2024, Art. no. 100353.

[7] E. Kuchumova, S. M. Martínez-Monterrubio, and J. A. Recio-Garcia, "STEG-XAI: Explainable steganalysis in images using neural networks," *Multimedia Tools Appl.*, vol. 83, no. 17, pp. 50601–50618, Nov. 2023.

[8] D. A. Shehab and M. J. Alhaddad, "Comprehensive survey of multimedia steganalysis: Techniques, evaluations, and trends in future research," *Symmetry*, vol. 14, no. 1, p. 117, Jan. 2022.

[9] M. R. Ayaluri, S. R. K., S. R. Konda, and S. R. Chidirala, "Efficient steganalysis using convolutional auto encoder network to ensure original image quality," *PeerJ Comput. Sci.*, vol. 7, p. e356, Feb. 2021.

[10] Y. Peng, Q. Yu, G. Fu, W. Zhang, and C. Duan, "Improving the robustness of steganalysis in the adversarial environment with generative adversarial network," *J. Inf. Secur. Appl.*, vol. 82, May 2024, Art. no. 103743.

[11] O. A. Alrusaini, "Deep learning for steganalysis: Evaluating model robustness against image transformations," *Frontiers Artif. Intell.*, vol. 8, Mar. 2025.

[12] A. H. B. Gezici and E. Sefer, "Deep transformer-based asset price and direction prediction," *IEEE Access*, vol. 12, pp. 24164–24178, 2024.

[13] T. Tuncer, U. Kaya, E. Sefer, O. Alacam, and T. Hoser, "Asset price and direction prediction via deep 2D transformer and convolutional neural networks," in *Proc. 3rd ACM Int. Conf. AI Finance*, Nov. 2022, pp. 79–86.

[14] N. Cassavia, L. Caviglione, M. Guarascio, G. Manco, and M. Zuppelli, "Detection of steganographic threats targeting digital images in heterogeneous ecosystems through machine learning," *JoWUA*, vol. 13, no. 3, pp. 50–67, 2022.

[15] N. Farooq and A. Selwal, "Image steganalysis using deep learning: A systematic review and open research challenges," *J. Ambient Intell. Humanized Comput.*, vol. 14, no. 6, pp. 7761–7793, Jun. 2023.

[16] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An image is worth 16×16 words: Transformers for image recognition at scale," 2020, *arXiv:2010.11929*.

[17] Y. Huo, K. Jin, J. Cai, H. Xiong, and J. Pang, "Vision transformer (ViT)-based applications in image classification," in *Proc. IEEE IEEE 9th Intl. Conf. Big Data Secur. Cloud (BigDataSecurity) Intl. Conf. High Perform. Smart Comput., (HPSC) IEEE Intl. Conf. Intell. Data Secur. (IDS)*, New York, NY, USA, May 2023, pp. 135–140.

[18] S. Wendzel, L. Caviglione, W. Mazurczyk, A. Mileva, J. Dittmann, C. Krätzer, K. Lamshöft, C. Vielhauer, L. Hartmann, J. Keller, and T. Neubert, "A revised taxonomy of steganography embedding patterns," in *Proc. 16th Int. Conf. Availability, Rel. Secur.*, Vienna, Austria, Aug. 2021, pp. 1–12.

[19] N. Ponomarenko, N. Gapon, V. Voronin, and K. Egiazarian, "Blind estimation of white Gaussian noise variance in highly textured images," in *Proc. 16th Image Process., Algorithms Syst.*, 2018, pp. 1–5.

[20] Y. Y. Ghadi, T. AlShloul, Z. I. Nezami, H. Ali, M. Asif, and M. J. Bah, "Enhanced payload volume in the least significant bits image steganography using hash function," *PeerJ Comput. Sci.*, vol. 9, p. e1606, Nov. 2023.

[21] S. Kang, H. Park, and J.-I. Park, "CNN-based ternary classification for image steganalysis," *Electronics*, vol. 8, no. 11, p. 1225, Oct. 2019.

[22] W. M. Eid, S. S. Alotaibi, H. M. Alqahtani, and S. Q. Saleh, "Digital image steganalysis: Current methodologies and future challenges," *IEEE Access*, vol. 10, pp. 92321–92336, 2022.

[23] D. R. Laxmi, "Advancements and challenges in image steganographer: A comprehensive review," *Int. J. Commun. Netw. Inf. Secur.*, vol. 15, no. 4, pp. 462–484, 2023.

ALANOUD M. ALMHLBDI is currently pursuing the M.S. degree in computer science with the University of Jeddah, specializing in computer science and artificial intelligence. Her research focuses on deep learning, image processing, and data protection using artificial intelligence and cybersecurity. She is developing a deep learning framework for image data analysis using convolutional neural networks and attention mechanisms to enhance data security and protect sensitive information in dynamic digital environments.

NORAH D. ALTOWAIRQI received the B.S. degree in computer science from Taif University, Saudi Arabia. She is a Graduate Researcher with the Department of Computer Science and Artificial Intelligence, University of Jeddah. Her current research focuses on integrating deep learning and computer vision techniques to detect steganographic content hidden in digital images, with the goal of enhancing the accuracy and resilience of secure data hiding systems. Her broader interests span artificial intelligence, cybersecurity, and the use of neural networks in digital forensics and intelligent security systems. In alignment with Saudi Vision 2030, her work aims to strengthen national capabilities in AI-driven digital protection and contribute to the development of secure and smart technologies for real-world applications.

AREEJ O. ALSHUTAYRI is an Associate Professor with the Department of Computer Science and Artificial Intelligence, University of Jeddah. She collected and created a social media Arabic dialect text corpus (SMADC) using Twitter, Facebook, and Online newspapers. Her research interests in using artificial intelligence, which includes machine learning and natural language processing, to understand languages, especially Arabic language and its dialects.

REHAB K. QAROUT is an academic Researcher, specializing in computer science and artificial intelligence technology. Her work focuses on the intersection of enhancing crowdsourcing tasks with natural language processing, information retrieval, image processing techniques, and optimization algorithms for improving computer efficiency. Her published research addresses learner performance evaluation and the design of technology-integrated instructional strategies that enhance learning outcomes. More recently, she has developed a strong interest in generative artificial intelligence and its applications in academic research, particularly in qualitative data analysis, knowledge generation, and informed decision-making in educational settings. Her work reflects a synthesis of rigorous research and emerging technologies, contributing to the advancement of the academic research field.